

# Projektdokumentation

## Entwicklung eines Backend-Systems für eine Festival-Bestellplattform

### 1 Einleitung

Im Rahmen dieses Projekts wurde eine Backend-Anwendung für eine digitale Bestellplattform für Festivals entwickelt. Ziel der Anwendung ist es, Besuchern eines Festivals die Möglichkeit zu geben, Speisen und Getränke digital über eine Anwendung zu bestellen. Gleichzeitig ermöglicht das System den Veranstaltern und Händlern eine strukturierte Verwaltung von Produkten, Bestellungen und Tickets.

Das Backend stellt eine zentrale REST-API bereit, über die ein Frontend mit der Datenbank kommunizieren kann. Dadurch wird eine klare Trennung zwischen Benutzeroberfläche und Geschäftslogik erreicht.

Die Implementierung des Systems erfolgte mit dem Java-Framework Quarkus, welches sich durch hohe Performance, hohe Stabilität und eine gute Integration moderner Java-Technologien auszeichnet. Für den Zugriff auf die Datenbank wurde das ORM-Framework Hibernate Panache eingesetzt, welches die Entwicklung von Datenbankzugriffsschichten vereinfacht.

### 2 Projektziel

Ziel des Projekts war die Entwicklung eines Backend-Systems, das folgende Anforderungen erfüllt:

- Bereitstellung einer REST-API für eine Festival-App
- Verwaltung von Accounts
- Verwaltung von Produkten und Foodständen
- Erstellung und Verwaltung von Bestellungen
- Verwaltung von Tickets
- Bereitstellung von Metadaten zu Produkten und Ständen (Bilder, Icons)
- Authentifizierung von Benutzern
- strukturierte API-Dokumentation

Darüber hinaus sollte die Anwendung eine saubere Architektur besitzen, die eine spätere Erweiterung und Wartung erleichtert.

# 3 Projektumgebung

Die Entwicklung erfolgte mit verschiedenen Technologien und Werkzeugen.

| Komponente                                                                                                                 | Technologie            |
|----------------------------------------------------------------------------------------------------------------------------|------------------------|
| Programmiersprache                                                                                                         | Java                   |
| Backend Framework                                                                                                          | Quarkus                |
| ORM                                                                                                                        | Hibernate Panache      |
| API Standard                                                                                                               | REST                   |
| API Dokumentation                                                                                                          | OpenAPI                |
| Versionsverwaltung                                                                                                         | Git / GitHub           |
| Datenbank                                                                                                                  | Postgre-SQL Datenbank  |
| Datenformat                                                                                                                | JSON                   |
| Frontend Integration                                                                                                       | TypeScript API Clients |
| Die Entwicklung wurde über ein Git-Repository verwaltet. Änderungen am System wurden regelmäßig über Commits dokumentiert. |                        |

# 4 Planung und Analyse

Zu Beginn des Projekts wurden zunächst die grundlegenden Anforderungen analysiert.

Dazu gehörten insbesondere:

- Definition der benötigten Systemfunktionen
- Modellierung der Datenbankstruktur
- Planung der API-Endpunkte
- Erstellung von UML-Diagrammen

Zur Visualisierung der Systemfunktionalität wurde ein Use-Case-Diagramm erstellt. Dieses beschreibt die Interaktion zwischen den Benutzern und dem System, insbesondere den Bestellprozess.

Darüber hinaus wurde ein Datenmodell erstellt, welches die Struktur der Datenbank und die Beziehungen zwischen den einzelnen Tabellen beschreibt.

Die Planung der API erfolgte ebenfalls frühzeitig, um eine klare Struktur für die spätere Implementierung zu schaffen.

# 5 Systemarchitektur

Die Architektur des Backends wurde in mehrere Schichten aufgeteilt.

Diese Trennung verbessert die Wartbarkeit und ermöglicht eine klare Verantwortungsstruktur innerhalb der Anwendung.

Die wichtigsten Schichten sind:

## **API Layer**

Der API Layer stellt die REST-Endpunkte bereit, über die das Frontend mit dem Backend kommuniziert. Die Endpunkte nehmen HTTP-Anfragen entgegen und geben JSON-Antworten zurück.

## **Service Layer**

Der Service Layer enthält die Geschäftslogik der Anwendung. Hier werden beispielsweise Bestellungen verarbeitet oder Benutzer authentifiziert.

Diese Schicht sorgt dafür, dass die API-Endpunkte möglichst schlank bleiben und die Logik zentral organisiert wird.

## **Repository Layer**

Der Repository Layer übernimmt den Zugriff auf die Datenbank. Hier werden Datenbankabfragen ausgeführt und Datenobjekte geladen oder gespeichert.

Durch die Verwendung von Hibernate Panache konnte dieser Layer stark vereinfacht werden.

## **Datenbank**

Die Datenbank speichert alle relevanten Informationen, wie beispielsweise:

- Benutzer
- Produkte
- Bestellungen
- Tickets
- Bilder

# **6 Datenbankdesign**

Für das Projekt wurde eine relationale Datenbankstruktur entwickelt.

Die wichtigsten Tabellen sind:

- Account  
Enthält Informationen über Benutzer des Systems.
- Product  
Speichert Produkte, die auf dem Festival verkauft werden.
- Order  
Repräsentiert Bestellungen von Benutzern.
- Ticket  
Stellt gekaufte oder registrierte Tickets dar.

Die Beziehungen zwischen den Tabellen wurden über Fremdschlüssel realisiert.

Um den Zugriff auf die Datenbank zu erleichtern, wurden zusätzlich Hibernate Entities erstellt. Diese bilden die Tabellen der Datenbank in Form von Java-Klassen ab.

## 7 Implementierung

Die Implementierung des Backends erfolgte schrittweise.

### 7.1 Grundstruktur des Backends

Zunächst wurde eine neue Quarkus Backend Anwendung erstellt.

In der ersten Phase wurden einfache Dummy-Endpunkte implementiert, um die grundlegende Funktionalität der API zu testen.

Anschließend wurde die Projektstruktur angepasst, um eine klare Trennung zwischen den verschiedenen Anwendungsschichten zu ermöglichen.

### 7.2 Datenbankintegration

Nachdem die Grundstruktur erstellt wurde, wurden Hibernate Entities implementiert. Diese ermöglichen einen einfachen Zugriff auf Datenbanktabellen.

Zusätzlich wurden Anfangs SQL-Skripte erstellt, mit denen die Datenbankstruktur automatisch erzeugt werden kann. Später wurde die Verwaltung der Datenbankstruktur Hibernate überlassen

Zur Vereinfachung der Entwicklung wurden außerdem Beispiel-Daten in die Datenbank eingefügt.

### 7.3 API Entwicklung

Im weiteren Verlauf wurden die verschiedenen REST-Endpunkte implementiert.

Zu den wichtigsten Endpunkten gehören:

- Produktabfragen
- Bestellungen erstellen
- Ticketverwaltung
- Bildverwaltung
- Benutzerlogin

Die API wurde kontinuierlich erweitert und refaktoriert, um eine saubere Struktur zu gewährleisten.

### 7.4 Refactoring

Während der Entwicklung wurden mehrere Refactoring-Schritte durchgeführt.

Dabei wurden unter anderem:

- die Projektstruktur angepasst
- Service-Klassen neu organisiert
- API-Endpunkte vereinheitlicht
- Repository-Klassen optimiert

Diese Anpassungen verbesserten die Wartbarkeit und Übersichtlichkeit des Codes.

## 7.5 Authentifizierung

Für die Benutzerverwaltung wurde eine Authentifizierungsfunktion implementiert.

Zunächst wurde eine Sessionverwaltung über Cookies verwendet.

Im späteren Verlauf wurde die Authentifizierung auf eine Header-basierte Lösung umgestellt, da diese besser für eine REST-API, die Geräteübergreifend angesprochen wird, geeignet ist.

Der Login-Endpoint liefert dabei eine JSON-Antwort mit den relevanten Informationen für die Authentifizierung.

## 7.6 Fehlerbehandlung

Um eine robuste Anwendung zu gewährleisten, wurde ein strukturiertes Fehlerbehandlungssystem implementiert.

Hierbei wurden layer-spezifische Exceptions eingeführt.

Diese ermöglichen eine bessere Kontrolle über Fehlerfälle und erleichtern das Debugging.

## 7.7 Logging

Zusätzlich wurde ein Logging-System implementiert, welches wichtige Ereignisse im System protokolliert.

Dies hilft dabei, Fehler schneller zu identifizieren und das Verhalten des Systems besser nachzuvollziehen.

Hierbei wurden die Log-Einträge auf 2 Dateien aufgeteilt, eine für eigenen Code und eine Datei für Quarkus-, Hibernate- und andere verwendete Librarys.

## 7.8 Erweiterte Funktionen

Im Laufe der Entwicklung wurden weitere Funktionen ergänzt.

Dazu gehören unter anderem:

- Endpoint Tracking
- Ticket-Verwaltung
- automatische Generierung von TypeScript API Clients

Die generierten TypeScript-Clients erleichtern die Integration der API in ein Frontend.

## 8 Testplanung

Für das Projekt wurde eine grundlegende Testplanung erstellt.

Dabei wurden insbesondere folgende Punkte getestet:

- korrekte Funktion der API-Endpunkte
- Authentifizierungsprozesse
- Datenbankzugriffe
- JSON-Antwortstrukturen

Die Tests erfolgten über API-Testtools sowie über das angebundene Frontend.

## 9 Dokumentation

Um die Nachvollziehbarkeit des Projekts zu gewährleisten, wurde eine umfassende Dokumentation erstellt.

Diese umfasst:

- OpenAPI API Dokumentation
- UML Diagramme
- Datenmodell
- Projektstruktur

Die Dokumentation erleichtert zukünftige Erweiterungen und die Einarbeitung neuer Entwickler.

## 10 Ergebnis

Im Rahmen des Projekts wurde ein funktionales Backend-System für eine Festival-Bestellplattform entwickelt.

Die Anwendung stellt eine strukturierte REST-API bereit, über die ein Frontend mit der Datenbank kommunizieren kann.

Durch die Verwendung moderner Technologien wie Quarkus, Hibernate Panache und OpenAPI konnte eine performante und wartbare Lösung implementiert werden.

Die entwickelte Architektur ermöglicht eine einfache Erweiterung des Systems und bietet eine stabile Grundlage für zukünftige Funktionen.